

Modelul Lambert – o singură culoare pentru un poligon al suprafeței

Modelul Gouraud – 1. se calculează câte o culoare pentru fiecare vârf al unui poligon IN VERTEX SHADER



2. culoile fragmentelor polig se calcul prin interpol între vârfuri

Modelul Phong – 1. se calculează câte o normală pentru fiecare vârf al unui poligon



2. normalele fragmentelor se calcul prin interp între normalele din vârfuri

---

2buffer pentru generarea umbrelor = SHADOW MAPPING

1. Generare shadow map (hartă a umbrelor) folosind un z-buffer, dar din perspectiva SURSEI DE LUMINĂ
   - valorile z calculate sunt stocate într-un z-buffer separat = shadow map
   - z = cea mai apropiată adâncime de suprafața luminată pentru fiecare punct din perspectiva sursei de lumină

   SCENA e desenată de cel puțin 2 ori

2. Utilizare shadow map pentru a determina umbrele
   - cu shadow map creat, se randează scena din nou din perspectiva OBSERVATORULUI
   1. Fiecare pixel vizibil din persp camerei, se calc poz acestuia în spațiul sursei de lumină
   2. Poz pixel CAMERĂ $\leq$ Poz în shadow map
   3. Dacă poz pixel CAMERĂ mai îndep decât valoarea în shadow map, înseamnă că există un obst direct între sursă și punctul respectiv, deci punctul e în umbră

În banda grafică, Explicați când se realizează DECUPAREA.

Decuparea primitivelor care sunt în afara volumului de vizualizare.
- se realizează după ce primitivele sunt transf. în spațiul observatorului (după ce sistemul a determinat unde ar trebui să apară pe ecran din perspectiva observatorului)
- și înainte de rasterizare
  - Rasterizare = primitivele decupate sunt „desenate" pe ecran, transformându-le din forme geometrice în pixeli

20

Translatia 2D

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{aligned} x' &= x + t_x \\ y' &= y + t_y \\ z' &= z + t_z \end{aligned}$$

Scalare 2D

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{aligned} x' &= S_x \cdot x \\ y' &= S_y \cdot y \\ z' &= S_z \cdot z \end{aligned}$$

Rotatie
Fata de origine

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos u & -\sin u & 0 \\ \sin u & \cos u & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$



In jurul Ox

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos u & -\sin u & 0 \\ 0 & \sin u & \cos u & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \begin{aligned} x' &= x \\ y' &= \cos u \cdot y - \sin u \cdot z \\ z' &= \sin u \cdot y + \cos u \cdot z \end{aligned}$$

In jur Oy

$$\begin{pmatrix} \cos u & 0 & \sin u & 0 \\ 0 & 1 & 0 & 0 \\ -\sin u & 0 & \cos u & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

In jur Oz

$$\begin{pmatrix} \cos u & -\sin u & 0 & 0 \\ \sin u & \cos u & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

1. Matrice Modelare, Vizualizare => Suntem în coord observator

2. Facem în coord observator – back-face culling
– frustum culling: eliminăm obiecte care nu se încadrează în volum de vizualizare
– decupare la limitele vol viz.: decupăm poligoanele care se intersectează cu marginile frustumului

3. Matrice de proiecție => Suntem în coordonate de decupare

4. Coord decupare: ∀ punct din scena 3D este transf. a.î. volumul de viz devine un ob canonic de dim 2×2, centrat în origine

5. Coord decupare —Împărțire la w→ Coord normalizate

6. Coord normalizate —Transf în Spațiu Ecran→ Spațiu Ecran

Decupare înainte de rasterizare

7. Rasterizare = Convertire în pixeli
Ray Tracing: după sau în locul rasterizării

8. Fragment Shader: – Se calc culoarea finală a fiecărui fragment, aplicând texturi, iluminare și umbre.
– Aici z-buffer: Aici facem z-ordering

Ray Tracing

1. Ray Casting: Pt fiecare pixel din imagine, se trasează o rază din poz camerei către scenă
2. Dacă raza cu obiectele scenă = Cel mai apropiat punct de pe
3. Știm punct intersecție. Culoare punct = pe baza material + lumini + umbre + reflexii + refracții
4. Reflexie, refracție: Dacă obiectul are proprietăți reflective sau refractive, se trasează raze adiționale de la I, raze R și T
5. Umbre: Se trasează raze de la I la sursa de lumină
Dacă această rază întâlnește un alt obiect, punct umbrit

Proiecții
- Persp
- Paralele (↓ Centru de proiecție la infinit)
  - ortogonale
    - axonometrice
    - vederi
    - = proiectori ⊥ plan proiecție
  - oblice = proiectori ✗ plan de proiecție dar paraleli între ei





MODELE DE CULOARE

- Modelul RGB
- Modelul CMY Cyan Magenta Yellow | $C = 1-R$, $M = 1-G$, $Y = 1-B$
- Modelul CMYK (black)

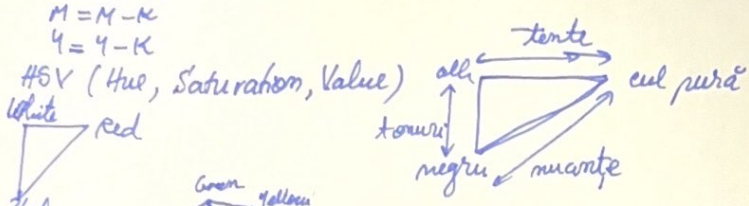
$K = \min(C, M, Y)$

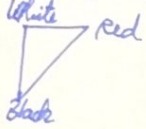
$C = C - K$

$M = M - K$

$Y = Y - K$

- Modelul HSV (Hue, Saturation, Value)







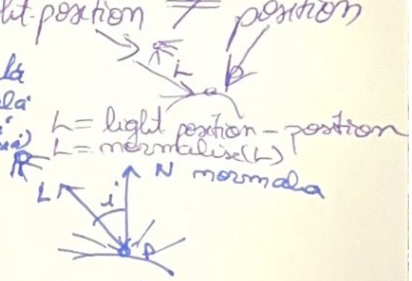
RGB	HSV
100 — Red	— 0, 1, 1
110 — Yellow	60, 1, 1
010 — Green	120, 1, 1

Iluminare locală ≠ globală

L = raza de lumină incidentă

R = rază reflectată

light position ≠ position



MODELE PT. CALCULUL LUMINII

- Modele empirice
  - Lumina ambiantă
  - Difuză
  - Speculară
  - Încandescență/Emisivă

  (în vertex shader)
- Modele analitice
  - Lambert
  - Gouraud
  - Phong
- Lumină
  - direcțională
  - pozițională (dist. finită față de scenă)

- Ambientală: $I_{am} = I_a \cdot K_a$ (constantă pt toate obiectele scenei); lumină distribuită uniform spațial
- Difuză (reflectată): $I_d = I_p \cdot K_d \cdot \cos(i)$ — $L \cdot N$ scalar product

  ↳ Influențează culoarea obiectului în funcție de poziția observatorului în scenă

  Factor atenuare

I_p = intens luminii sursă

K_d = coef de difuzie a luminii incidente = constanta obiectului asociată = prop de material

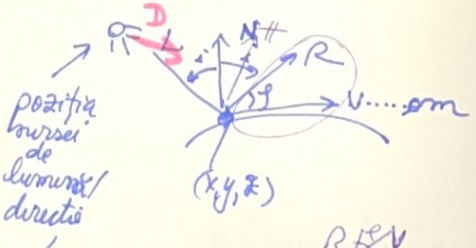
- Specular: dă strălucire
  - văd strălucire numai pe direcția observatorului

$$I_p = I_p \cdot K_s \cdot \cos(\varphi)^n \quad (R \cdot N)^n$$

I_p depinde de material, intensitate asociată sursei

K_s = coef de reflexie specular

n = coef de strălucire



$I_d = f_{atenuare} \cdot I_p \cdot K_d \cdot \cos(i)$

$I_s = I_p \cdot K_s * (R \cdot V)^n = I_p \cdot K_s \cdot (N \cdot H)^n$

Experimental $H = (L+V)$ ⇒ Numai e nev. să calculăm R

$N \cdot H \simeq R \cdot V$

R ≠ N



Proiecții { Perspectivă ⇐ / Paralelă ⇐ } ⇒ Observator (față de planul de proiecție) { la o dist FINITĂ / INFINITĂ }

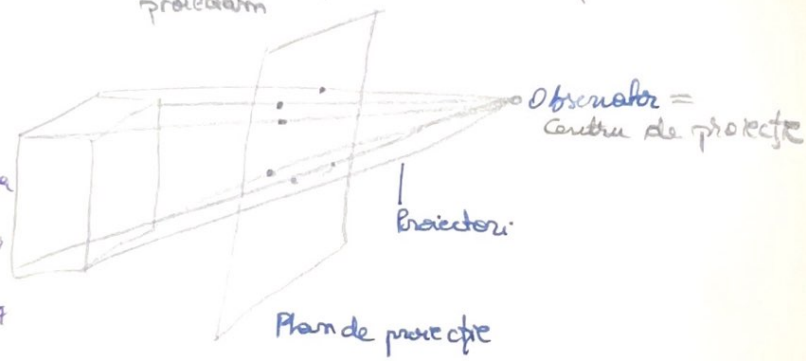
Centrul de proiecție = observator

Proiectorii care trec prin centrul de proiecție

Proiectori perpendiculari pe planul de proiecție/proi. care nu sunt perp pe plan de proiecție

Perspectivă
Observ dist finită] Unesc observator cu vârfurile obiectului pe care-l proiectam

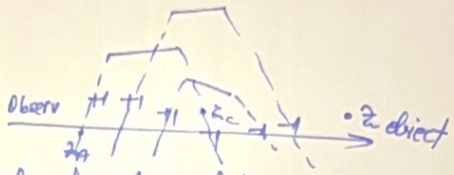
- img realistă
- dimensiuni nu sunt păstrate
- Nu se păstr paralelismul între dreptele
- Liniile paralele care nu sunt paralele cu planul de proiecție se intersectează într-un PUNCT de CONVERGENȚĂ



Paralelă
Obs la ∞] Proiectori paralelă cu <u>direcția de proiecție</u>

Proiecții
- Poziționare centru de proi
  - Persp (finită)
    - 1 pct convergență
    - 2 pct conv
    - 3 pct conv
  - Paralele (infinit) — față de plan de proiecție
    - Ortogonale (Vederi) = văd o sg față ortografice: x, y același; z=0 constant
      - Rotesc obiectul / mut planul de pr: axonometrice
    - Oblice: Proiectori neperp cu plan pr
      - Cavalier
      - Cabinet
- Transf rigide
- Matrice de proiecție
  - Instanțiere + proiecție
    - 4×4
    - 4×4
  - (x, y, z, 1) coord omogene

Volume de umbră

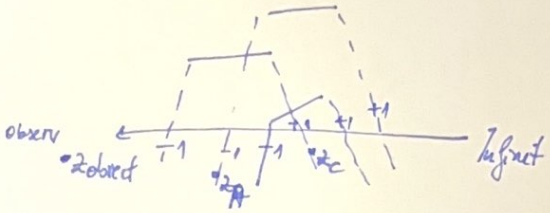


Z PASS: Modificăm contor dacă z pass

1. Simulez fețele către observator ale volumelor de umbră
2. Dacă $z_A < z_{obiect}$ [nu modific z-buffer
   contor +1
3. Simulez fețele spate
4. Dacă $z_c < z_{obiect}$ [nu modific z-buffer
   contor --

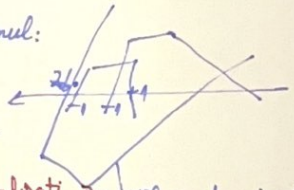
Pentru contor fiecare fragment, folosim stencil buffer.

Z-FAIL: Modifică contor dacă z FAIL



1. Simulez fețele spate
2. Dacă $z_c > z_{obiect}$ (z fail) — contor++
3. Simulez fețe față
4. Dacă $z_A > z_{obiect}$ (z fail) — contor--

- Plan spate e infinit ca să evităm situații de genul:
  Rămânem cu −1, dar obiectul nu e în volum de umbră.
  ~~Inițiem contor cu nr de volume care conțin poz observatorului.~~
  In depth-pass, dacă observator e în volum de umbră, este greu de calculat mai ales în câte volume de umbră e observatorul.



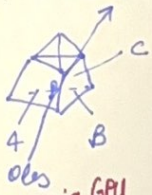
- Clasificarea poligoanelor în PUV și PUN se face pe baza [normalelor
  [și poziției observatorului

- Un punct P este umbrit dacă vectorul observator P intersectează mai multe poligoane PUV decât PUN.
  PUV +1
  PUN −1
  • P umbrit dacă valoarea contor e pozitivă în P

back face culling

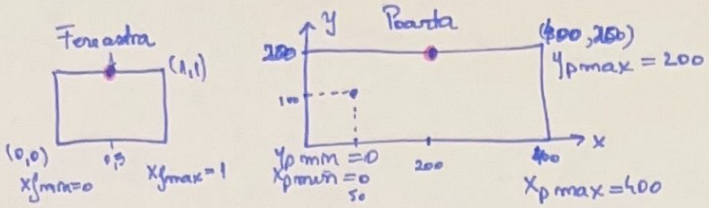
Contor = Valori din stencil buff sunt dif de 0 pt pixelii care trb afișați în volum de umbră.
Stencil buffer = Utilizat pt a implementa diferite operații grafice, de regulă în combinație cu z-buffer.
Valori stencil buffer increm/decrem pentru fiecare fragment care trece/nu trece testul de adâncime

Poligoane de umbră vizibile (PUV)
PUV — Poligon vizibil din poziția observatorului
− A, B



Poligoane de umbră nevizibile (PUN)
PUN — Poligon nevizibil din poziția observatorului
− C

In memoria GPU
||
Pe GPU se încarcă grafică
Stencil buffer = buffer de marcaje
Frame Buffer = buffer imagine
Buffer Adâncime = z-buffer



A (0,5 ; 1) → Cale pe m poartă pt o scalare uniformă afișată în contrat în poartă

$s = \min(s_x, s_y)$

$$\begin{cases} t_{s_x} = \dfrac{(x_{pmax} - x_{pmin} - s\,(x_{fmax} - x_{fmin}))}{2} \\ t_{s_y} = \text{la fel pt } y_p, y_f \end{cases}$$

$(0,5;1)\cdot 100 = (50,100)$
Dar final (200,200)

$$s_x = \frac{x_{pmax} - x_{pmin}}{x_{fmax} - x_{fmin}} = \frac{400-0}{1-0} = 400$$

$$s_y = \frac{y_{pmax} - y_{pmin}}{y_{fmax} - y_{fmin}} = \frac{200}{1} = 200$$

$\Big\}\ s = 200$

s_x, s_y factorii de scalare ai transformării

$t_x = x_{pmin} - s_x \cdot x_{fmin}$ $\quad T_x = t_x + t_{s_x}$ $\quad t_x = 0 - 200\cdot 0 = 0$

$t_y = y_{pmin} - s_y \cdot y_{fmin}$ $\quad T_y = t_y + t_{s_y}$ $\quad t_y = 0 - 200\cdot 0 = 0$

t_x, t_y depind de poziția celor 2 ferestre față de originea sist de coord

$$t_{s_x} = \frac{400 - 0 - 200\cdot(1-0)}{2} = 100$$

$$t_{s_y} = \frac{200 - 0 - 200\,(1-0)}{2} = 0$$

Translația totală — Translația suplimentară pentru afișare centrată în poartă

x_p, y_p

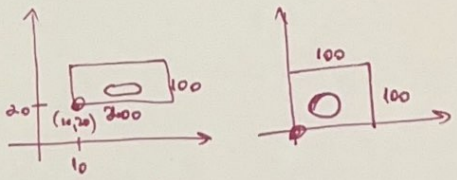
$$x_p = \underset{0,5}{x_f} \cdot s_x + t_x + t_{s_x} = 0,5\cdot 200 + 0 + 100 = 100 + 100 = 200$$

$$y_p = \underset{1}{y_f} \cdot s_y + t_y + t_{s_y} = 1\cdot 200 + 0 + 0 = 200$$

$\Big\}\ A'(200,200)$

Deci, $s_x = \dfrac{\text{lungime poartă}}{\text{lungime fereastră}}$

$s_y = \dfrac{\text{lățime poartă}}{\text{lățime fereastră}}$



$t_x = 0 - \frac{1}{2}\cdot 10 = -5$

$t_y = 0 - 1\cdot 20 = -20$

$s_x = \dfrac{100}{200} = 0,5$

$s_y = \dfrac{100}{100} = 1$

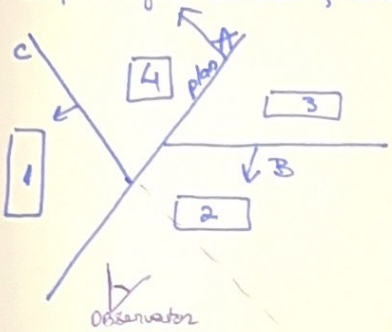
Afișare centrată în poartă: translație suplimentară pe axa O_x sau pe axa O_y

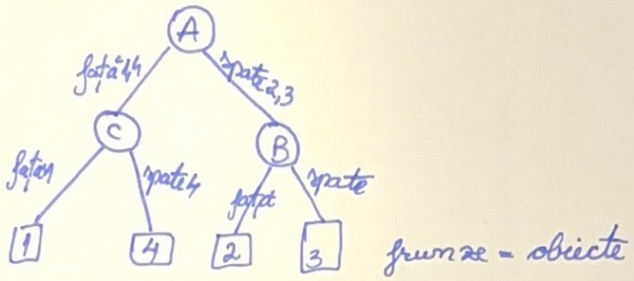
Translația totală: $t_x + t_{s_x}$

$t_y + t_{s_y}$

ALGORITMUL de PARTIȚIONARE BINARĂ A SPAȚIULUI
BSP

1. arbore
2. parcurgere arbore ținând cont de poz. observatorului





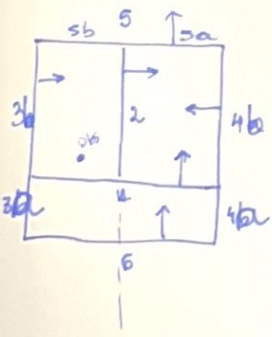
frunze - obiecte

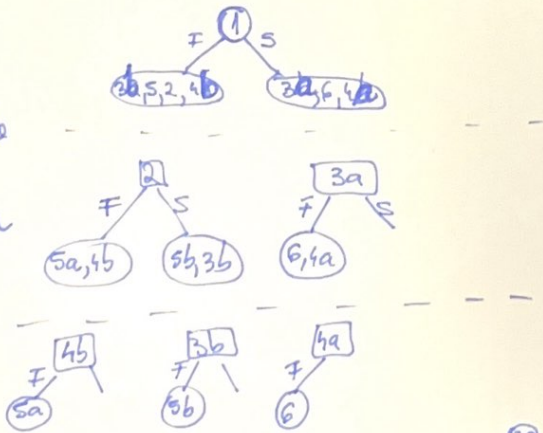
1. Fac Arbore
2. Construiesc scena — Poziția Observatorului?

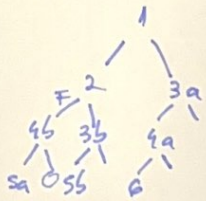
Plan A → Unde e obs? → în spatele → Mă uit la subarbore față
Plan C → Unde e obs? → în față → Mă uit la spate ⇒
Plan B → unde e obs $\xrightarrow{4,1}_{3,2}$ → în față → Mă uit în spate ⇒

⇒ 4, 1, 3, 2

- Când modific arbore? De fiecare dată când se modifică scena.
- Modif poziției obs. nu modifică arborele. ⇒ Se modif afișarea/parcurgerea.







3a, 4a, 6, 4, 4b
↓
5a, 2, 3b, 5b

Plan 1 ⇒ Obs în față ⇒ Mă uit la subarb spate



Plan 3a ⇒ Obs în față ⇒ Mă uit la subarb spate ⇒ Afișez 3a
4a ⇒ Obs față ⇒ Mă uit spate ⇒ Nu am ⇒ Afișez 4a
6 ⇒ Obs față ⇒ Spate
2 ⇒ Obs spate ⇒ Mă uit față
4b ⇒ Obs față ⇒ Mă uit spate ⇒ Nu am ⇒ Afișez
Mă uit față Afișez

Specific: → Volum de vizualizare, Eliminare obiecte: frustum culling

Sistem de coord propriu/local/ al unui obiect

aceeași pt tot obiectul

M de modelare

Sistem de coord real/global/ al lumii

accesat per obiect

M de Vizualizare

lookAt(poz, dir, orientare)

Se def volum vizualiz când sunt deja aia CU LOOKAT

Sistem de coord de vizualizare = Spațiu observator = Spațiu asociat camerei

Se def volum de vizualizare specific tip de proiecție

Sistemul camerei

ca să rămân numai cu obiectele care sunt vizibile de observator

Translație + Aliniere de axe = Transf ale sistemelor de coordonate

::ortho(l, r, b, t, n, f)

paralel

::perspective(fovy, aspect, n, f) = ::frustum(l, r, b, t, n, f)

unghi / w/h

camera: v, u, n

Transf de proiecție

Volum vizual canonic = Viewport 3D coordonate decupare Cub centrat în origine de latură 2 (x_3, y_3, z_3, w)

Proiecție ortogr de tip vedere → Ecran

Păstrează numai x, y; z se reține la el

3D → 2D

• Înglobează proiecții perspectivă + Decupare (Eliminare a obiectelor care nu sunt în volumul de vizualizare)

Trb coord carteziene, nu omogene ca mai înte

Împărțire la w

Împărțire perspectivă

Transf fereastră-poartă = glViewport()

• w s-ar putea să nu mai fie 1

Sistem real/global al lumii

glu: lookAt(, ,)
- poziție (un punct)
- direcția de privire (un punct)
- orientare: vector up

Observatorul privește spre camerei, să zicem z negativ (n) spre obiecte

Camera în sist de VIZUALIZARE = ÎN ORIGINE privește către −z

1. Poziția camerei în spațiu global
2. Direcția de privire − vector Look − normala ↑ negativ
3. Vector Up = Oy în sist observ

openGL fol coord omogene

$(x, y) \to (x, y, 0, 1)$

$u = Up \times n = y \times z = x$

$v = n \times u = z \times x = y$

$\begin{pmatrix} x_3 \\ y_3 \\ z_3 \\ w \end{pmatrix} = PVM \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$

Implicit observ este:
- în origine
- privește de-a lungul −z
- up = y+

Ray Tracing

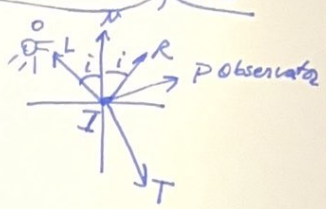
Pentru fiecare pixel al imaginii

1. Se calc raza care porneste de la observator si trece prin pixel dun suprof de afisare in planul imaginii
2. Se obs punct intersecte I al razei primare cu cel mai apropiat obiect de observator

Se calculează cul. obiectului în punctul I, ținând cont:

Contribuție 3 tipuri de raze emisă, R, T

1. Lumina de la sursă L
2. Lumina prov de la alte obiecte pe dir reflexiei speculare R
3. Lumina în I prov de la alte obiecte pe dir razei transmise refract T



- recursiv

Îmbunătățire: Volume încadratoare și gruparea obiectelor din scenă

- Efectuat DUPĂ sau IN LOCUL etapei de rasterizare

Intensitatea luminii descreste proporțional cu inversul pătratului distanței de la sursa de lum la obiect

- Cele 2 metode pentru componenta speculară: a) utilizarea razei reflectate a luminii incidente
  b) utilizarea vectorului H

Care e mai rapidă/eficientă? $\frac{1}{d^2}$

$I = I_{emisă} + I_{amb} + f_a(I_{difuză} + I_{speculară})$

$I_S = I_{sursă} \cdot K_S \cdot (R \cdot V)^n$ Phong

$I_S = I_{sursă} \cdot K_S \cdot (N \cdot H)^n$ Blinn-Phong



H poate fi calculat rapid; este mai puțin sensibil la schimbările de poziție ale observatorului ceea ce reduce computațional costul; calcul H este mai rapid, nu necesită calculul reflexiei propriu zise

$H = \frac{L+V}{\|L+V\|} \Rightarrow$ normalizare

$I_{difuză} = I_{sursă} \cdot K_d \cdot (L \cdot N)$

$I_{amb} = I_{ambiental\ global} \cdot K_a$

Lumina ambientală are vine dintr-o direcție anume, ea apărând ca și cum ar veni din toate direcțiile. Este INDEPENDENTĂ de poziția sursei de lumină.

Componenta difuză = lumina reflectată de suprafața obiectului în mod egal în toate direcțiile

Componenta speculară = lumină reflectată de suprafața obiectului numai în jurul direcției $\vec{R}$
= depinde și de poziția observatorului